

Fail-Closed Admission Control for Autonomous Agent Coordination

Mechanising Trust Boundaries in Multi-Agent LLM Systems

Author: William Vaughn

Date: August 28, 2026

Repo: <https://github.com/bv-public-cc/agent-coordination-substrate>

Abstract

Multi-agent systems built on large language models are being deployed with production authority - mutating repositories, actuating devices, and assigning work to one another - on coordination layers designed for cooperating processes rather than for actors that fail in the ways language models fail. An agent reports success whether or not it succeeded. Message ingestion is unobservable at the transport layer. Success conditions arrive as prose that may be incapable of failing. The coordination record is writable by the same agents it holds to account.

This paper argues that the coordination layer, not the model, is the correct place to enforce these boundaries, and that enforcement must fail closed: absence of affirmative evidence must produce refusal, not permission. It presents seven mechanisms implementing that principle - dispatch admission gated on falsifiability of success criteria, decision-divergence admission, replay-receipt delivery confirmation, reference-intent integrity triage, digest-bound publication, process-lineage sender authentication, and observation-gated claim reclamation - with source, execution transcripts, and evaluation against a production system.

Three results are reported. On a 1,687-event coordination ledger carrying 1,977 cryptographic bindings, 180 bindings no longer matched their recorded digests; an undifferentiated integrity check reports all 180, whereas classifying each binding by an intent declared at publication and sealed inside the immutable record reduces this to 11 items requiring action - a 94% reduction in triage load. On a constructed corpus of forty acceptance criteria, lexical screening for unfalsifiable success conditions achieved recall 1.00 at precision 0.95, with the single false positive arising from self-referential quotation. And a commit adding a correct, well-tested admission predicate silently invalidated both the project's executable demonstration and its primary documentation while 131 unit tests remained green, because nothing asserted that the documented path still executed.

That third finding names a defect class - *verification claim drift* - argued to be the dominant residual risk in assurance-oriented systems. A mechanised control is presented, together with the account of its own first failure. Every command and figure in this paper is reproducible from a clean clone; Appendix A gives the exact sequence.

1. Introduction

1.1 The deployment reality

Systems that place several language-model agents under an orchestrating agent have moved from demonstration to production. An orchestrator decomposes an objective and assigns work; implementer agents mutate repositories or actuate devices; reviewer agents verify candidates. Coordination occurs through a shared substrate - a filesystem, a version-controlled repository, a message bus - that every participant reads and writes.

The security posture of such a system is usually discussed in terms of what an agent is permitted to invoke. Considerable recent work addresses this: permission frameworks for agent skills (SkillGuard, 2026), architectural patterns separating capability expression from authority grant (2026), threat taxonomies for skill architectures (2026), and capability-based authorization in inter-agent protocols (A2A Project, 2026). This work is necessary and it is not sufficient. It governs the *verbs* available to an agent. It does not govern whether the *work assigned* is work whose completion can be judged, whether an instruction actually arrived, or whether the record of what happened can still be trusted.

Those three questions are the subject of this paper.

1.2 Why the model is the wrong enforcement point

A recurring instinct is to address these problems through prompting - instruct the agent to be careful, to verify its work, to report honestly. This fails for a structural reason unrelated to model capability: the artifact being asked to enforce the constraint is the same artifact the constraint exists to bound. An agent that has misunderstood its task will report success against that misunderstanding with complete sincerity. An agent instructed not to proceed without confirmation will, under sufficient context pressure, proceed and describe having confirmed.

This is the argument that moved input validation from client to server. Enforcement must live where the constrained party cannot reach it.

1.3 Fail-closed as the governing principle

Every mechanism here is structured so that **absence of affirmative evidence produces refusal**. This is more aggressive than the conventional default. In most systems a missing field is a warning, an unreachable check is skipped, and an ambiguous state resolves toward continuing. In an autonomous agent system those defaults are inverted in their consequences: the work proceeds unattended, at machine speed, and the operator learns the outcome afterward.

Fail-closed carries a real cost, and this paper does not minimise it. A gate that refuses too often is disabled by the next engineer it obstructs, and a disabled gate protects nothing. Section 8 addresses the design constraints this imposes, which prove as important as the mechanisms themselves, and Section 7.3 measures one of them.

1.4 Contributions

1. A threat model for LLM multi-agent coordination identifying four failure modes not addressed by capability- or permission-based controls (§3).
2. Seven fail-closed coordination mechanisms, presented at implementation depth with source and execution transcripts, each independently usable (§4).
3. Quantitative evaluation of integrity-signal triage on a production ledger: a 94% reduction in items requiring human attention (§7.1).

4. Quantitative evaluation of lexical falsifiability screening: recall 1.00, precision 0.95 on a constructed corpus, with analysis of the failure mode (§7.3).
 5. Identification of *verification claim drift* as a distinct defect class, with a case study in which it defeated a 131-test suite, a mechanised control, and an account of that control's own first failure (§7.2).
 6. Discussion of gate design constraints - precision over recall, refusal messages as a control surface, redundant enforcement - grounded in observed failure (§8).
 7. A complete reproduction path from a clean clone (Appendix A).
-

2. Related work

Multi-agent LLM coordination. Surveys of collaboration mechanisms (Tran et al., 2025) identify coordination failures, ambiguous handoffs, and redundant communication as central open problems. Architectural work has converged on persistent shared state with audit. HearthNet (2026) deploys role-specialised agents coordinating through MQTT and Git-backed shared state, with root-issued actuation leases binding each state-changing command to an identity, device scope, parameter envelope, and the state version under which it was approved, plus a non-participant observer agent mirroring coordination traffic for crash recovery and auditability. Event-sourcing approaches (ESAA, 2026) apply replay verification with hashing to establish immutability of completed tasks. Schema-grounded state mutation (PatchBoard, 2026) constrains how agents modify shared state.

These systems address durability and auditability of what agents *did*. The present work addresses admission - whether the work should have been dispatched, whether the instruction arrived, and whether the audit record's own signal remains interpretable at scale. HearthNet's leases are short-lived and reclaimed on expiry with a fresh request; §4.7 argues that expiry-based reclamation is unsafe for agents specifically, and presents an alternative.

Agent authorization. SkillGuard (2026) presents a permission framework for agent skills. Architectural pattern work (2026) articulates that skills should be requests for capability rather than grants of authority, with execution authority checked separately, and that agents should request capabilities scoped to the task rather than the session. Threat taxonomy work (2026) analyses the attack surface of skill architectures. Capability-based authorization is under active discussion in inter-agent protocols (A2A Project, 2026), drawing on classical capability security in which a capability is an unforgeable token designating a resource and authorizing an operation.

This literature governs invocation. It does not evaluate the judgeability of an assigned task, which §4.1 addresses.

Integrity and tamper evidence. Efficient tamper-evident logging (Crosby & Wallach, 2009) established performance foundations for hash-chained audit logs, extended to constrained environments via Merkle pipelines with adaptive chunking (2026). These answer whether a log was altered, treating every recorded reference as equally immutable.

File-integrity monitoring products - Tripwire, AIDE and comparable - do classify detected change as expected or suspect, explicitly to suppress false positives. This is the closest prior work to §4.4 and is distinguished in detail there.

Delivery confirmation. Acknowledgment and at-least-once delivery are standard in message-oriented middleware, including agent-specific exchanges (LLM-X, 2026), which specifies an Ack message type with retry

under TTL. The end-to-end argument (Saltzer, Reed & Clark, 1984) established that correctness properties must be enforced at the endpoints that care about them rather than in transport. Section 4.3 applies that argument one layer further than is conventional: in an agent deployment, the process wrapping the model is still transport.

Liveness and exclusive access. Lease arbitration with time-based reclamation is standard. High-availability clustering rejects timeout-only reclamation, resolving ambiguity through fencing - STONITH and comparable mechanisms isolate or power off a suspect node before takeover. Section 4.7 adopts the premise and rejects the resolution.

3. Threat model

The adversary here is primarily *non-malicious*. This is a deliberate scoping choice. The dominant risk in deployed agent systems is not an attacker subverting an agent; it is a capable, well-intentioned agent operating confidently on a wrong premise, at machine speed, unattended. Controls designed for the malicious case frequently do not catch this, because nothing about the agent's behavior is anomalous.

T1 - Self-reported success is not evidence

A conventional process that fails returns a non-zero status. An agent asked to complete a task will, in the failure case, frequently report completion in fluent and confident prose. Any control consuming the agent's own account of its work as input to an acceptance decision inherits this.

T2 - Ingestion is unobservable at the transport layer

Writing bytes to an agent process establishes that bytes were written. It does not establish that a turn was constructed, that the model attended to the content, or that the instruction entered working context. A message can be written successfully and never acted upon, indistinguishably from success. Standard acknowledgment does not help: the ACK is emitted by the process wrapping the model, whether or not the model saw anything.

T3 - Success conditions may be incapable of failing

Work is assigned in prose, and prose admits conditions nothing can contradict: *verify the module works correctly, ensure output is reasonable, handle errors gracefully*. An agent given such a criterion will report satisfying it, correctly, because the criterion cannot be falsified. The defect is in the order rather than the agent, and it is undetectable at the point of acceptance - the moment when review attention is actually available.

This failure mode has the worst economics of the four. The cycle is spent, the review is performed, and both produce nothing, because the acceptance question was unanswerable before work began.

T4 - The coordination record is agent-writable, and its integrity signal decays

Agents write to the same substrate that records what they did. Malformed serialization, duplicate identity claims, pointers to absent content, and retroactive edits are all reachable from ordinary agent behavior without adversarial intent.

Worse, a record binding cryptographic digests of referenced artifacts accumulates mismatches through legitimate activity: a draft is revised, a report regenerated, a working file changes after being referenced. An integrity check reporting every mismatch produces a defect list dominated by expected change. Section 7.1 quantifies this at 94% noise, and a control at that signal-to-noise ratio is not triaged - it is disabled, which returns the system to having no integrity control while appearing to have one.

4. Mechanisms

Each mechanism is presented with its enforcement point, implementation, refusal condition, and distinguishing property. All are independently usable. Source is Python 3.9+ with PyYAML as the only external dependency.

4.1 Falsifiability-gated dispatch admission

Enforcement point: before any agent process is started.

A structured decomposition of the objective is evaluated against machine-checkable predicates. The validator returns issuable, refused-with-enumeration, or unreadable. A refusal names every failed predicate rather than short-circuiting, because an author correcting one defect at a time iterates against the gate rather than reconsidering the decomposition.

The validator never executes any command named within the decomposition. Confirmation is the planner's act. A validator that executed content from a planning document would itself become an injection surface - a document authored by an agent, executed by a privileged verifier, is precisely the shape to avoid.

The predicate addressing T3 screens acceptance criteria for falsifiability:

```
# Acceptance language that cannot fail. Narrow on purpose: a checker with false
# positives on ordinary wording gets argued with, then disabled.
UNFALSIFIABLE = [
    (re.compile(r"\bworks? (?:(?:correctly|properly|as expected|fine)\b", re.I),
     "works correctly"),
    (re.compile(r"\b(?:is|are|looks?) (?:(?:good|fine|clean|reasonable|sensible)\b",
     re.I),
     "is good"),
    (re.compile(r"\bas (?:(?:appropriate|needed|necessary)\b", re.I),
     "as appropriate"),
    (re.compile(r"\bhandles? .{0,20} (?:(?:properly|gracefully|appropriately)\b",
     re.I),
     "handles gracefully"),
    (re.compile(r"\bno (?:(?:major|significant|obvious)
(?:issues|problems|regressions)\b", re.I),
     "no major issues"),
    (re.compile(r"\bimproved?\b(?:!.*\b(?:by|to|from)\s)", re.I),
     "improved, with no measure"),
]
```

The last pattern is the subtlest: `improved` is admissible when followed by a measure or baseline (`improved by 40% from the July figure`) and inadmissible alone. The negative lookahead encodes that distinction.

Applied at admission:

```
if task.get("essential") is True:
    essential_count += 1
    acceptance = str(task.get("acceptance", "")).strip()
    if not acceptance:
        problems.append(f"{task_id} is essential but has no acceptance criterion;
"
                        f"an essential task without one cannot be judged
complete")
    else:
        for pattern, label in UNFALSIFIABLE:
            if pattern.search(acceptance):
                problems.append(f"{task_id}: acceptance criterion is unfalsifiable
"
                                f"({label!r}); state what is observed and how")
                break
```

The remaining predicates enforce: the mission statement answers who, what, when, where and **why** - purpose separately required, because an agent encountering unanticipated conditions and unable to reach its orchestrator has no basis for choosing between stopping and guessing unless purpose is stated; every asserted fact about system state names a confirming command *and* an observation obtained from it; every declared information requirement enumerates at least two outcomes mapping to at least two *distinct* subsequent actions; every capability required by an essential task is sourced to an assigned element; every external asset grant names an identity digest, enumerates exact operations, refuses wildcards, and attaches to a declared task; required and prohibited actions do not intersect; planning does not exceed one third of allotted time; and a restatement by the receiving party is required before execution.

The information-requirement predicate is worth showing, because it turns a question that reads as judgment into a computation:

```
for index, entry in enumerate(opord.get("ccir") or []):
    outcomes = entry.get("outcomes")
    if not isinstance(outcomes, list) or len(outcomes) < 2:
        problems.append(f"{label!r}: fewer than two outcomes; an information "
                        f"requirement that cannot change a decision is not
critical")
        continue
    actions = {normalize(o.get("next_action", "")) for o in outcomes if
isinstance(o, dict)}
    actions.discard("")
    if len(actions) < 2:
        problems.append(f"{label!r}: every outcome leads to the same action; "
                        f"take the action instead of collecting the information")
```

Distinguishing property. Schema validation and policy-as-code check structural conformance. This evaluates a semantic property of natural language - whether a stated success condition is capable of being false - and gates dispatch on it.

4.1.1 Execution transcript

A decomposition containing nine deliberate defects (`refused.json`, Appendix A), submitted for admission:

```
$ python3 decompose_check.py refused.json
Decomposition refused:
mission statement has no 'why'; a subordinate without the purpose cannot
exercise initiative when the plan meets reality
'CI runs on every push': no confirmed_by command. An assumed fact is not
reconnaissance, and this is where live cycles die.
T-1: acceptance criterion is unfalsifiable ('works correctly'); state what is
observed and how
T-2: acceptance criterion is unfalsifiable ('improved, with no measure');
state what is observed and how
no implied tasks and no rationale. Doctrine excludes routine and SOP work from
implied tasks, so an empty list is legitimate -- but it must be a stated
finding, not an unasked question.
'run the full test suite' appears as both a constraint and a restraint
'Does the ledger contain duplicate identities?': every outcome leads to the
same action; take the action instead of collecting the information
planning takes 90 of 120 minutes (75%); at most one third belongs to planning,
the rest to execution
backbrief_required must be true; a subordinate restating the mission costs one
round trip and catches a misdirected cycle

$ echo $?
1
```

Two features of this output are design choices rather than presentation. First, all nine defects report together: a gate that reports one defect per invocation trains the author to satisfy it iteratively rather than reconsider the decomposition. Second, every message states an operational consequence rather than naming a violated rule - the reasoning behind §8.2.

The same decomposition with all nine corrected:

```
$ python3 decompose_check.py issuable.json
Issuable: 2 tasks, 2 essential, 1 CCIR, 0 support grant(s), 0 element(s),
layer=platform-build.
Now run the companion checks:
--emit ccir | outcome_gate.py - (each requirement changes a decision)
--emit slices | ticket_graph.py - (blocking edges, one state per ticket)

$ echo $?
0
```

The correction that mattered most is not visible in the exit code. The original **T-1** acceptance read "*the validator works correctly and handles malformed input gracefully*" - a criterion an agent could report satisfying under any outcome. The corrected form reads "*publishing an assignment without credential_contract exits non-*

zero with 'assignment event requires credential_contract'. The second can fail; the first cannot. No amount of downstream review recovers that difference, because review of an unfalsifiable criterion has nothing to check against.

4.2 Decision-divergence admission

Enforcement point: publication of an escalating record.

An event escalating an ambiguity is refused unless it carries a decision contract declaring a class from a closed set, a question, the next material decision, and a reversible default. Where the class is ambiguity, four further predicates apply:

```
if decision_class != "ambiguity":
    return
if contract.get("next_action_reversible") is not False:
    raise PublishError("reversible ambiguity must proceed under a recorded
assumption")
check = contract.get("bounded_check")
if not isinstance(check, dict):
    raise PublishError("ambiguity requires bounded_check")
for field in ("method", "budget", "stop_condition"):
    value = check.get(field)
    if not isinstance(value, str) or not value.strip():
        raise PublishError(f"bounded_check.{field} is required")
outcomes = contract.get("outcome_actions")
if not isinstance(outcomes, list) or len(outcomes) < 2:
    raise PublishError("ambiguity requires at least two outcome_actions")
actions = set()
for item in outcomes:
    ...
    actions.add(action.strip().lower())
if len(actions) < 2:
    raise PublishError(
        "ambiguity does not change the next action; proceed without escalation")
```

Three properties deserve comment.

`next_action_reversible is not False` is a three-state test written as two. A contract asserting `true`, and a contract omitting the field entirely, both refuse. Only an explicit `false` admits escalation. This is fail-closed applied to a boolean: silence is not consent, and an agent that omits the field does not thereby acquire permission.

The semantics are the reverse of the intuitive reading. Escalation is admitted only where the next action is *irreversible*. A reversible ambiguity must proceed under the recorded default rather than stopping to ask - because for an agent operating unattended, an escalation costs a full round trip through a human, and paying that for a decision that could be cheaply undone is the more common waste.

The final predicate is the same test §4.1 applies to information requirements, enforced by independent code over a different data shape. This redundancy is intentional; §8.3 develops the reasoning.

4.3 Replay-receipt delivery confirmation

Enforcement point: advancement of the acknowledged-sequence watermark.

Each wake message carries an opaque transport identifier. The listener writes the message and records it outstanding. **The watermark does not advance:**

```
def dispatch_if_new(self) -> bool:
    """Primary rail: a newer validated orchestrator pointer."""
    pointer = read_authoritative_pointer(self.topology, self.role)
    if pointer["sequence"] <= self.last_sequence:
        return False
    if self.pending is not None:
        return False
    transport_id, message = _pointer_message(self.role, pointer)
    self._send(message)
    self.pending = {**pointer, "kind": "pointer",
                    "transport_id": transport_id,
                    "sent_monotonic": self.clock()}
    self._write_state("awaiting-transport-ack")
    return True
```

Note what `_send` does not do: it does not touch `self.last_sequence`. The watermark advances in exactly one place in the codebase:

```
def accept_output(self, line: bytes) -> bool:
    """Consume the replay receipt that proves the turn was ingested."""
    try:
        value = json.loads(line)
    except (TypeError, ValueError):
        return False
    if self.pending is None:
        return False
    if self.pending["transport_id"] not in _text_from_message(value):
        return False
    if self.pending["kind"] == "pointer":
        self.last_sequence = self.pending["sequence"]
    self.pending = None
    self._write_state("running")
    return True
```

The receipt is read from the agent CLI's replay stream - the same channel carrying the model's generated output. Three conditions must hold: the line parses, a message is outstanding, and the outstanding transport identifier appears within the *content the model emitted*. Only then does `last_sequence` advance.

A message written to the process but never ingested therefore leaves the watermark unchanged, and the same work is presented again on the next cycle rather than being silently lost. If no receipt arrives within a bounded interval:

```
def assert_not_stale(self) -> None:
    if self.pending and self.clock() - self.pending["sent_monotonic"] >
self.ack_timeout:
        self._write_state("transport-ack-timeout")
        raise BridgeError(
            f"listener did not acknowledge wake within {int(self.ack_timeout)}
seconds")
```

It raises. It does not proceed as though delivery occurred, and it does not retry silently - a persistent non-delivery is an operator-visible fault, not a background condition.

Distinguishing property. The confirming evidence is generated by the language model, in the same output channel that carries its work product. This is the end-to-end argument (Saltzer et al., 1984) applied one layer beyond convention: in an agent deployment the process wrapping the model is still transport, and a transport-generated ACK does not address T2.

4.3.1 Wake origination and self-scheduling containment

Four rails may originate a wake, at most one outstanding, each independently deduplicated: **inbound pointer** (primary); **continuation**, where the role's own last record names a next pause point and carries no stop signal; **staleness**, where orchestrator state still declares the role active and nothing arrived within a window; and **monitor error**, where orchestrator state cannot be read, signalled once per interval and deduplicated by a digest of the error text.

Rails two through four all begin identically:

```
def dispatch_self_wake_if_due(self) -> bool:
    """Stale-timer rail, admitted only by the orchestrator-owned ledger."""
    if self.pending is not None:
        return False
    try:
        if not role_has_active_work(self.topology, self.role):
            self.heartbeat_monitor_error = None
            return False
```

`role_has_active_work` reads the **orchestrator-owned** ledger. An agent cannot originate its own continued scheduling, because the predicate admitting a self-originated wake is written by a party the agent cannot write as. This closes a containment gap that is easy to create accidentally: an agent that can schedule itself runs indefinitely without operator action, and will, if a loop condition is misjudged.

The continuation rail additionally refuses on any explicit stop signal:

```
if (
    progress.get("task_paused") is True
    or progress.get("response_required") is True
    or progress.get("decision_required") is True
    or progress.get("terminal_decision") is not None
```

```

    or progress.get("lock_released") is True
):
    return False

```

The monitor-error rail deserves note as a design pattern. When the ledger cannot be read, the auxiliary scheduler cannot determine whether the role is active - a condition that could reasonably either halt the listener or be ignored. It does neither:

```

except BridgeError as exc:
    # The auxiliary scheduler never takes down the proven pointer rail.
    # Its observation defect is persisted and signalled through the child
    # at most once per stale interval, without task authority.
    self.heartbeat_monitor_error = str(exc)

```

The defect is surfaced to the agent, deduplicated by error digest so a persistent fault does not produce a wake storm, and explicitly carries no task authority. The primary rail continues operating throughout. A degraded auxiliary control that disables a working primary control is a net loss in availability for no gain in safety.

Every wake message declares that transport conveys no scope, task, retry, or acceptance authority. Scheduling is not instruction; a receipt is not consent.

4.4 Reference-intent integrity triage

Enforcement point: the composite fail-closed indicator gating CI.

Every digest binding recorded within a coordination record declares an intent:

- **immutable_ref** - must always verify; a mismatch is an integrity failure.
- **snapshot_at_publication** - a point-in-time record of an artifact expected to change; a mismatch is expected drift.

The intent is declared **by the writer at publication** and stored **inside the append-only record**. It is therefore exactly as tamper-evident as the digest it qualifies: reclassifying a reference to suppress an alarm requires altering a record whose alteration is itself detected.

The discriminator:

```

def check_bindings(root: Path, records: Iterable) -> tuple[int, list]:
    """Recompute every declared reference and classify what disagrees."""
    for record in records:
        for binding in _declared_bindings(record.body):
            intent = binding.get("reference_intent")
            ...
            if intent not in (IMMUTABLE_REF, SNAPSHOT_AT_PUBLICATION):
                defects.append(Defect(UNKNOWN_INTENT, rel, f"intent={intent!r}",
record.path))
                continue

```

```

target = root / rel
if target.is_symlink():
    defects.append(Defect("symlinked-target", rel, "", record.path))
    continue
try:
    actual = hashlib.sha256(target.read_bytes()).hexdigest()
except OSError:
    kind = ("immutable-failure" if intent == IMMUTABLE_REF
           else "snapshot-absent")
    defects.append(Defect(kind, rel, "target absent", record.path))
    continue
if actual == expected:
    continue
if intent == IMMUTABLE_REF:
    defects.append(Defect("immutable-failure", rel,
                          "content differs from the recorded digest",
                          record.path))
else:
    # Expected: the file was always going to change after this
    # publication recorded it. Reported, never alarming.
    defects.append(Defect("snapshot-drift", rel,
                          "changed since publication", record.path))

```

An unrecognised intent is itself a defect, so classification cannot be evaded by omission - a binding that declines to declare is not thereby exempt. A symlinked target is refused without being read, closing a redirection through the reference itself.

The composite indicator:

```

@property
def fail_closed(self) -> bool:
    """True when something happened that a reader must not ignore.

    Snapshot drift is expected and never trips this. Unparseable events,
    immutable-reference failures, and duplicate identities do.
    """
    return bool(self.rejected or self.immutable_failures or
                self.duplicate_identities)

```

Records load through a parser that rejects duplicate mapping keys rather than silently retaining the last - a YAML default that would otherwise let a second `sha256:` key override the first inside a single record.

4.4.1 Execution transcript

The mechanism is directly observable at small scale. `make integrity-demo` builds a ledger with one binding of each intent, then edits each in turn:

```

$ make integrity-demo
Ledger built: 1 event, 2 bindings (1 immutable, 1 snapshot)

```

```

--- baseline: nothing changed ---
- reference bindings checked: **2**
- immutable failures: **0**
- snapshot drift (expected): **0**
**fail_closed: false**
exit: 0

--- after editing the SNAPSHOT reference (expected drift) ---
- reference bindings checked: **2**
- immutable failures: **0**
- snapshot drift (expected): **1**
**fail_closed: false**

1 snapshot reference(s) changed since publication. That is the declared intent
and is not a defect.
exit: 0

--- after editing the IMMUTABLE reference (integrity failure) ---
- reference bindings checked: **2**
- immutable failures: **1**
- snapshot drift (expected): **1**
**fail_closed: true**

## Defects requiring attention

- `immutable-failure` `accepted-result.txt` - content differs from the recorded
  digest (referenced by `bridge/events/...-1-boss-task-accepted.yaml`)
exit: 1

Both assertions held: drift is reported without alarming;
an immutable mismatch fails closed.

```

The two middle cases are the mechanism. Identical operations - a file's bytes change after a record bound its digest - produce opposite outcomes, determined solely by what the writer declared at publication. Exit status follows: the snapshot edit leaves CI green, the immutable edit breaks the build.

Distinguishing property. File-integrity monitoring achieves noise suppression through administrator-authored policy that is external to the monitored data, applies per path, and is independently mutable. Here classification is per *reference* - the same artifact may be bound immutably by one record and as a snapshot by another, because intent describes the reference rather than the file

- declared by the writer at the moment of binding, and sealed inside the immutable record. An external policy can be edited to silence an alarm. This cannot be.

4.5 Digest-bound publication

Enforcement point: installation and pointer replacement.

Publication proceeds under a single exclusive lock spanning the entire critical section:

```

# Orchestrator routes share one sequence space and therefore one explicitly
# declared lock. A derived target would move when a role is added.
lock_target = (
    topology.root / topology.global_sequence_lock
    if pointer_rel in topology.orchestrator_routes
    else pointer_path
)

# The lock covers both the monotonicity check and replace. Without it, two
# valid concurrent publications can both validate against the same old
# pointer and the lower sequence can replace the higher sequence last.
with _pointer_lock(lock_target):
    _validate_agent_sender(pointer_rel, topology)
    _validate_global_orchestrator_sequence(pointer_rel, spec.get("event", {}),
    topology)
    event, pointer, event_path, pointer_path, event_rel = _validated_inputs(spec,
    topology)
    event_body = _dump(event)

    event_tmp = event_path.parent / f".{event_path.name}.{os.getpid()}.
    {uuid.uuid4().hex}.tmp"
    _write_fsynced(event_tmp, event_body)
    try:
        # A hard link installs the fully written event atomically and
        # refuses overwrite if another publisher already claimed the name.
        os.link(event_tmp, event_path, follow_symlinks=False)
    except FileExistsError as exc:
        raise PublishError("append-only event already exists") from exc
    finally:
        event_tmp.unlink(missing_ok=True)
    _fsync_directory(event_path.parent)

    placed = event_path.read_bytes()
    if yaml.safe_load(placed) != event:
        raise PublishError("installed event failed read-back validation")
    digest = hashlib.sha256(placed).hexdigest()

    final_pointer = dict(pointer)
    final_pointer["authoritative_event"] = {"path": event_rel, "sha256": digest}

```

Six properties are load-bearing.

The lock target switches on route class. Orchestrator outbound routes share one monotonic sequence space, so they must share one lock; a per-route lock would let two orchestrator publications race the cross-route sequence check. A previous draft derived the lock path from the lexically first inbound pointer, which would silently *move* the lock when a role was added - briefly leaving two locks live and reopening the race the lock exists to close. It is now an explicit topology field.

The digest is computed from `placed`, not from `event_body`. These are the same bytes under normal operation, and the distinction is the entire point: the digest binds what is on disk, not what was intended to

be. A caller-supplied digest is refused outright rather than trusted, because the party with an interest in the binding may not assert it.

Installation uses `os.link` with `follow_symlinks=False`. The link refuses an existing name, making append-only a property of the filesystem rather than of convention or of an agent's cooperation.

Read-back validation re-parses the installed bytes and compares to the intended object before the digest is taken, so a partial write cannot be bound.

Serialization round-trips before install:

```
def _dump(value: dict) -> bytes:
    body = yaml.safe_dump(value, sort_keys=False, default_flow_style=False,
width=100)
    body = body.encode("utf-8")
    if yaml.safe_load(body) != value:
        raise PublishError("YAML round-trip validation failed")
    return body
```

Prose containing a colon, a leading dash, or a tab cannot produce an authoritative record that later fails to parse. Given that record bodies contain agent-authored natural language, this is not a theoretical concern - the production ledger contains two events that predate this check and are permanently unparseable (§7.1).

The ordering is asymmetric. The event installs before the pointer is replaced, so an interrupted publication can leave a valid record that no pointer references - inert and detectable. It cannot produce a pointer referencing absent, invalid, or mismatched content. Failure degrades toward an orphan, never toward a dangling reference.

The demonstration exercises the monotonicity guarantee directly:

```
$ make demo
1. published event bridge/events/20260801T120000Z-1-boss-task-assigned.yaml
2. pointer digest matches installed bytes (f6178e36f12f...)
3. refused non-monotonic sequence: sequence is not greater than the global
   orchestrator sequence
```

Step 2 recomputes the digest of the installed file and asserts equality with the pointer's recorded binding - the property §4.4 later depends on.

4.5.1 Class-specific admission

Certain record classes carry additional predicates. A halt-shaped record must declare its blast radius:

```
def _validate_pause_scope(event: dict) -> None:
    """Require explicit blast-radius analysis before a workflow can stop.

    Ambiguous safety language previously allowed a local integrity concern to
```

```

halt an entire task. Stop-shaped events must identify the exact boundary
and the work that continues. A task-wide stop needs positive overlap proof;
an empty continuation list is never accepted by omission.
"""

...
for field in ("blocked_action", "shared_state", "last_verified_state",
             "continuation_lanes", "resume_condition"):
    value = scope.get(field)
    if value is None or value == "" or value == {}:
        raise PublishError(f"pause_scope.{field} is required")
lanes = scope["continuation_lanes"]
if not lanes:
    justification = scope.get("task_wide_stop_justification")
    proof = scope.get("overlap_proof")
    if not isinstance(justification, str) or not justification.strip():
        raise PublishError("task-wide stop requires
task_wide_stop_justification")
    if not isinstance(proof, list) or not proof:
        raise PublishError("task-wide stop requires non-empty overlap_proof")

```

An empty continuation set is the task-wide case, and it is never accepted by omission - a stop that halts everything must argue for its scope. The docstring records the incident this closed.

A work-assignment record must declare credential handling. The `none` mode is strict about its own emptiness:

```

mode = contract.get("mode")
if mode == "none":
    if set(contract) != {"mode"}:
        raise PublishError("credential_contract mode none permits no interfaces")
    return

```

A null contract cannot quietly carry an exception. Where credentials are conveyed, each interface must declare an environment variable name that is a valid identifier, a scope, an approved transport, exposure prohibitions covering at minimum `{argv, logs, events, artifacts, helper-body}`, and prohibited names - and is refused outright if it carries a literal secret:

```

if {"value", "token", "secret", "password"}.intersection(interface):
    raise PublishError("credential interface must never contain a secret value")

```

The contract conveys where a credential may be obtained and where it must never appear. It never conveys the credential.

4.6 Process-lineage sender authentication

Enforcement point: acceptance of an agent-originated publication.

A role lock does not authenticate a process: a second process can reuse the same session identifier and assert the same writer fields.

```
def _validate_agent_sender(pointer_rel, topology, *, ancestors=None) -> None:
    """Refuse agent-authored events outside the registered listener lineage.

    A role lock alone does not authenticate a process: another process can
    reuse the same agent session and claim the same writer/instance fields.
    Binding outbound publication to the durable listener's child-process
    ancestry makes that duplicate process unable to publish under the
    listener's identity.
    """
    role_name = topology.agent_routes.get(pointer_rel)
    if role_name is None:
        return
    state = json.loads((topology.root / role.listener_state).read_text())
    child_pid = state.get("child_pid")
    status = state.get("status")
    if not isinstance(child_pid, int) or child_pid < 2:
        raise PublishError(f"{role_name} listener child identity is invalid")
    if status not in {"running", "awaiting-transport-ack"}:
        raise PublishError(f"{role_name} listener is not in a publishing state")
    lineage = _process_ancestors() if ancestors is None else ancestors
    if child_pid not in lineage:
        raise PublishError(
            f"caller is outside the registered {role_name} listener process
            lineage")
```

The direction matters and is easy to invert when describing: the listener's recorded *child* process must appear in the *caller*'s ancestry. The publisher must be a descendant of the registered agent process, not the reverse. Ancestry is walked through `/proc/<pid>/stat`, terminating at init or on any read failure.

This binds write authority on a coordination route to process lineage rather than filesystem permission - meaningful precisely where several agent processes execute under one operating-system account, which is the common deployment. It does not defeat a local attacker with equivalent privilege, and does not claim to; it raises the cost of a specific accidental failure in which a second agent process, legitimately started, publishes under an identity it does not own.

4.7 Observation-gated claim reclamation

Enforcement point: reclamation of an expired exclusive claim.

The claim is acquired by atomic directory creation and bound to a tuple of task identity, attempt, request digest, and owner:

```
try:
    lease_dir.mkdir(mode=0o700)
except FileExistsError:
    current = _read(lease_path)
```

```

_assert_owner(current, task_id=task_id, attempt=attempt,
               request_sha256=request_sha256, owner=owner)
return {"action": "claim-replayed", "lease": current}

```

An exactly matching tuple returns a **replay indication** rather than an error and rather than a second grant, so a re-presented wake does not duplicate committed work. This is what makes at-least-once delivery (§4.3) safe: the same instruction delivered twice produces one execution. A differing tuple refuses, which the demonstration exercises:

5. claimed lease, expires 2026-08-03T15:40:13Z
6. refused second owner: lease owner tuple differs
7. released lease; slot is free

Owner identity is a process number paired with its start time:

```

# A pid alone is not an identity, because pids are reused. Pairing it with
# the process start time makes a dead owner distinguishable from a live
# stranger wearing its number. The keys are omitted rather than written null
# when /proc cannot answer: absence means "never captured", which reap()
# treats as unadjudicable, and a null identity on disk would be a claim we
# did not establish.
for field, pid in (("listener_start_ticks", listener_pid),
                  ("listener_child_start_ticks", listener_child_pid)):
    ticks = process_start_ticks(pid)
    if ticks is not None:
        value[field] = ticks

```

The omit-rather-than-null decision is small and consequential. Writing `null` would assert that identity was captured and found empty; omitting the key preserves the distinction between *unknown* and *known-absent*, and reclamation depends on it.

Reclamation requires expiry **conjoined with** observed absence:

```

def reap(topology, *, proc_root=Path("/proc"), clock=None) -> dict:
    """Release the lease only when its owner is provably gone.

    Refuses in every ambiguous case: no lease, owner alive, or an owner whose
    identity was never captured. The refusal reason is returned rather than
    raised, because a reaper that cannot decide is a normal outcome and not an
    error.
    """
    record = read_lease(topology)
    if record is None:
        return {"action": "reap-declined", "reason": "no-lease-present"}

    alive = owner_is_alive(record, proc_root)
    if alive is None:

```

```

        return {"action": "reap-declined", "reason": "owner-identity-not-
captured",
                "detail": "the lease predates start-time capture; "
                "it can expire but not be reaped", "lease": record}
    if alive:
        return {"action": "reap-declined", "reason": "owner-alive", "lease":
record}

    expires = _parse_stamp(record.get("expires_at"))
    if _now(clock) < expires:
        return {"action": "reap-declined", "reason": "owner-gone-but-lease-
unexpired",
                "detail": "an owner may die and be resumed inside its own lease
window",
                "lease": record}

    path.unlink(); _fsync_directory(lease_dir); lease_dir.rmdir()
    return {"action": "reaped", "reason": "owner-gone-and-lease-expired", "lease":
record}

```

Condition	Outcome
No claim present	Declined
Owner identity never captured	Declined - may expire, never reclaimable
Owner observed alive	Declined
Owner absent, claim unexpired	Declined - an owner may die and resume within its window
Owner absent and expired	Reclaimed

Declines are **returned with a specific reason, not raised**. A reclaimer that cannot decide is a normal outcome, not an error condition, and modelling it as an error produces alert fatigue against a control operating correctly.

Refresh is separately gated:

```

if _parse_stamp(value.get("expires_at")) <= instant:
    raise LeaseError("expired lease cannot be refreshed")

```

An expired claim cannot be extended back to life; recovery requires explicit release or adjudicated reclamation.

Distinguishing property. High-availability clustering also rejects timeout-only reclamation, but resolves the ambiguity by fencing - taking affirmative action against the suspect node to *create* the safety it then relies on. Here no action is taken against the holder; the mechanism observes, and where observation is impossible it declines permanently rather than escalating. The "never captured, therefore never reclaimable" state has no fencing analogue, because fencing always has a further action available.

5. Implementation

The reference implementation is approximately 3,500 lines of Python across five modules plus a dispatch validator, with PyYAML as the only external dependency.

Module	Lines	Responsibility
<code>publisher.py</code>	727	Digest-bound publication, class admission, sender auth
<code>listener.py</code>	837	Wake transport, replay receipt, four rails, supervision
<code>lease.py</code>	460	Exclusive claim, replay, observation-gated reclamation
<code>integrity.py</code>	318	Reference-intent verification, duplicate identity
<code>topology.py</code>	305	Declarative bridge description
<code>decompose_check.py</code>	483	Dispatch admission (separate repository)

Exclusivity relies on POSIX `mkdir`, `flock`, and `link` atomicity on a single filesystem. The implementation is explicitly not NFS-safe and does not claim to be a distributed system.

A declarative topology document names the orchestrator, roles, routes, event directories, ledger, claim location, and global sequence lock:

```
schema_version: 1
orchestrator: boss
orchestrator_events_dir: bridge/events
ledger: state.yaml
lease_dir: bridge/locks/mutating.lock

defaults:
  events_dir: bridge/events
  listeners_dir: bridge/listeners

roles:
  worker:
    inbound_pointer: bridge/boss-to-worker.yaml
    outbound_pointer: bridge/worker-to-boss.yaml
    holds_mutating_lease: true
    active_work:
      task_path: [execution, active_task]
      pause_path: [execution, safe_pause]
```

No module holds a hard-coded role name, path, or route, so retargeting is a configuration change rather than a code change. The integrity verifier additionally knows no event type names at all, which is what allows it to port across deployments unchanged - throughput measurement, which necessarily encodes an event vocabulary, is kept in a separate module for exactly this reason.

Delivery is at-least-once by design; idempotency is the caller's responsibility via a task key, backed by the claim replay semantics of §4.7. The system is not a message queue, not a permission system, and not an agent

framework - it carries authority between agents that already exist.

6. Method

Evaluation used three approaches.

Production measurement (§7.1). The integrity verifier was executed against a live coordination ledger accumulated over approximately five weeks of continuous multi-agent operation on a real software delivery workload. No records were constructed for the evaluation and none were excluded. This measurement is not reproducible by a third party, since the ledger is not public; §4.4.1 gives a self-contained demonstration of the same mechanism at small scale, and Appendix A gives the command.

Constructed corpus evaluation (§7.3). Falsifiability screening was measured against a forty-item labelled corpus, twenty unfalsifiable and twenty falsifiable. This corpus was author-constructed, which is a material limitation discussed in-line: it measures the screen against its own design intent rather than against field data. Field measurement was attempted and proved impossible for an instructive reason reported in §7.3.

Incident analysis (§7.2). The verification-drift finding was not planned. It emerged from attempting to execute the system's own documented procedures, and is reported as a case study with the remediation and the remediation's own subsequent failure.

7. Results

7.1 Integrity signal quality

```
$ python3 -m coordination_substrate.integrity \
  --root <ledger-root> --event-dir coordination/bridge/events --format json
```

Measure	Value
Records loaded	1,687
Digest bindings checked	1,977
Unparseable records	2
Immutable-reference failures	11
Snapshot drift (expected)	169
Unrecognised intent	0
Duplicate (writer, sequence) identities	2
Composite fail_closed	true

180 bindings no longer matched their recorded digests. An undifferentiated integrity check reports all 180 with no means of separating tampering from routine revision. Intent classification resolves this to **11 items requiring action** - a 94% reduction in triage load.

The operational significance is not the arithmetic. A control reporting 180 findings of which 169 are expected is not triaged; it is switched off, or its output is skimmed, which is equivalent. The intent split is the difference between an integrity control that is *used* and one that is nominally present.

Three secondary observations, each with a transferable lesson.

Unrecognised intent is zero. Every one of 1,977 bindings declared a recognised intent. This is the predicate from §4.4 that reports an undeclared binding as itself a defect, working as designed: because omission is not an escape, the population is fully classified rather than partially.

The duplicate identities are historical. Both duplicate (*writer*, *sequence*) pairs date from before the cross-route sequence check was introduced, separated by 40 and 80 minutes respectively - far too long to be a lock race. The mechanism is correct; the ledger carries scars from before it existed. This is worth naming: a fail-closed control introduced into a running system will surface a backlog, and treating that backlog as evidence against the new control is a common way to lose it. The correct response is adjudication of the historical records, not modification of the check.

Several immutable failures are misclassification, not tampering. Inspection of the 11 shows entries such as *coordination/tools/publish_bridge_event.py* and its test file, bound as *immutable_ref* and subsequently edited in the ordinary course of development. Source files change; binding them immutably guarantees a permanent false alarm. The correct remediation is reclassification to *snapshot_at_publication*, not investigation - and the mechanism surfaces the distinction cleanly enough to make that determination cheap.

This last point is a caveat on the headline result and is stated plainly: the 94% reduction measures separation of *declared-immutable* from *declared-snapshot*, not separation of tampering from non-tampering. Where a reference is misclassified at publication, the mechanism faithfully reports the consequence of that misclassification. It improves signal; it does not substitute for correct declaration.

7.2 Verification claim drift: a case study

The second finding was not planned. It emerged from attempting to verify the system's own documented behavior, and is arguably more transferable than the first.

A commit introduced a new admission predicate: work-assignment records must carry a credential contract (§4.5.1). The predicate was correct, deliberately designed, and accompanied by thorough unit coverage. It also silently invalidated two artifacts the commit did not touch.

Running the project's advertised end-to-end demonstration:

```
$ make demo
bridge root: /tmp/tmp88dvtiql

Traceback (most recent call last):
  File "scripts/demo.py", line 55, in main
    result = publish(
  File "coordination_substrate/publisher.py", line 310,
    in _validate_task_assignment_credentials
    raise PublishError("assignment event requires credential_contract")
coordination_substrate.publisher.PublishError:
```

```
assignment event requires credential_contract
make: *** [Makefile:11: demo] Error 1
```

The primary documentation - a walkthrough that builds a bridge from an empty directory and runs a task through it - contained the same defect: zero occurrences of `credential_contract`, and a `task-assigned` publication as its first executable step.

Both the README and the provenance document asserted that *every command in the walkthrough had been executed against this code*. From the moment of that commit, the assertion was false.

The test suite remained entirely green - 131 tests passing in 12.0 seconds. Coverage of the new predicate was thorough. Nothing covered the path a reader actually takes.

This is a distinct defect class, and naming it is useful: **a verification claim outlived the thing it verified**. The code was never wrong. The statement *about* the code silently became wrong, and because such statements live in prose rather than in executable form, nothing detected it. In a system whose value proposition is mechanised assurance, an unverified assurance claim is the most consequential possible defect, because it is the one that causes a reader to trust something they should not.

7.2.1 Remediation

A test was added asserting that the documented surface still executes. It runs the demonstration and requires exit zero, and it extracts every publish payload from the documentation and checks each against the publisher's own validators - **in both directions**:

```
if expects_refusal:
    # The doc shows a REFUSED line. If the validators now accept
    # it, the walkthrough is teaching a guarantee that no longer
    # holds -- as wrong as breaking a working example.
    self.assertIsNotNone(
        refusal,
        f"documented event {label!r} shows a REFUSED line but is "
        f"now accepted; the guarantee it teaches has been lost.")
else:
    self.assertIsNone(
        refusal,
        f"documented event {label!r} is refused by {refusal}. "
        f"Either the walkthrough needs updating or the validator "
        f"is too strict.")
```

The second direction matters as much as the first. Documentation that teaches a guarantee which has quietly stopped holding is at least as harmful as documentation that fails to run - arguably more so, since a broken example is noticed immediately and a hollow guarantee is not.

7.2.2 The remediation's own first failure

Writing that test surfaced a further instructive error, reported because it illustrates the same defect class recurring one level up.

The first version asserted that every documented payload must pass validation. It failed:

```
AssertionError: documented event '2-safety-stop' is refused by
_validate_pause_scope: stop-shaped event requires pause_scope.
```

The test was wrong; the documentation was right. The walkthrough *deliberately* publishes an invalid record to demonstrate a refusal, annotated with the expected error:

```
python3 -m coordination_substrate.publisher /tmp/stop.json --root $BRIDGE
# REFUSED: stop-shaped event requires pause_scope
```

The corrected version reads that annotation and asserts the documented outcome rather than a universal one. A verification control written on an assumption about the thing it verifies reproduces exactly the defect it was built to catch.

The test file additionally carries negative controls, on the principle that a check which cannot fail is not evidence:

```
class ParserControlTests(unittest.TestCase):
    """Negative control: a test that cannot fail is not evidence.

    If ``_fenced_json_blocks`` silently stopped matching, every assertion above
    would pass vacuously over an empty list. These prove it still discriminates.
    """
```

This mirrors an existing control in the concurrency suite, `test_the_race_harness_actually_detects_a_missing_lock`, which re-runs a lock race with the lock patched out and asserts the harness notices.

7.2.3 Resulting state

```
$ make test
Ran 140 tests in 11.753s

OK

$ make demo
1. published event bridge/events/20260801T120000Z-1-boss-task-assigned.yaml
2. pointer digest matches installed bytes (f6178e36f12f...)
3. refused non-monotonic sequence: sequence is not greater than the global
   orchestrator sequence
4. refused assignment with no credential contract: assignment event requires
   credential_contract
5. claimed lease, expires 2026-08-03T15:40:13Z
6. refused second owner: lease owner tuple differs
```



```
7. released lease; slot is free
```

```
all demo assertions held
```

Step 4 is the addition: the demonstration now *asserts* the predicate that broke it, so the guarantee is something the demo proves rather than something it trips over.

The project's own provenance document was simultaneously found to assert 104 tests where 140 existed, and to omit an entire 24-test module from its per-file table - the same defect class in miniature, in the document that exists to record verification.

7.3 Falsifiability screening: precision and recall

Field measurement was attempted first and proved impossible, for a reason that is itself a result.

The production ledger contains no corpus of natural-language acceptance criteria to measure against. Task requests are structured as executable assertions - `command` paired with `expected_exit_code`, appearing 47 times across the task population - rather than as prose. The system had already converged on the strongest available form of falsifiability, which is executable acceptance. This supports the paper's thesis while denying it field data, and is reported rather than worked around.

Measurement therefore used a constructed corpus of forty acceptance criteria: twenty unfalsifiable, twenty falsifiable. **This corpus was author-constructed, so it measures the screen against its design intent rather than against independently sampled field data. The figures should be read accordingly.**

Measure	Value
Unfalsifiable set	n = 20
- detected	20
- missed	0
Falsifiable set	n = 20
- false positives	1
- correctly passed	19
Recall	1.00
Precision	0.95

The single false positive is instructive:

FALSE POSITIVES:

```
'works correctly' <- the corpus flags 'works correctly' and not  
                      'works around the lock'
```

The flagged criterion is one that *quotes* the offending phrase while describing the screen itself. This is a self-reference artifact rather than a semantic failure - the screen is lexical and cannot distinguish use from

mention. It is also the realistic shape of the false-positive class: acceptance criteria for work *on* the checker will trip the checker.

Two conclusions. First, at this corpus the screen's precision cost falls on a narrow and recognisable class, supporting the design argument in §8.1. Second, recall of 1.00 on a corpus built to the patterns' own design intent is close to tautological and should not be read as a general detection claim; the honest statement is that the screen reliably catches the class it was built for, and the open question - what fraction of field-occurring unfalsifiable criteria fall inside that class - remains unmeasured.

8. Discussion

8.1 Precision over recall in gate design

The falsifiability corpus detects a deliberately narrow class. It does not attempt to catch every unfalsifiable acceptance criterion, and a determined author evades it trivially by rephrasing.

This is a design decision, not a limitation to be engineered away. A screen producing false positives on ordinary phrasing will be argued with, then bypassed, then removed - and a removed gate protects nothing, having also consumed the political capital needed to introduce a better one. The achievable goal is to render a *curated class* mechanically inadmissible at near-zero false positive cost, and to expand the corpus only as evidence accumulates.

The §7.3 measurement supports this: the one false positive in twenty falls on self-referential quotation, a class an engineer encountering it would recognise as a quirk rather than as the gate being wrong. That distinction - between a false positive that reads as a bug and one that reads as an obstruction - largely determines whether a control survives.

The general principle: **for a control that humans can disable, the false-positive rate is a survival characteristic, not merely a quality metric.**

8.2 Refusal messages as a control surface

Every refusal states why in operational terms rather than naming a violated rule. "An unsourced essential task refuses the launch rather than warning about it." "A grant with no task behind it is standing authority nobody asked for." "An assumed fact is not reconnaissance, and this is where live cycles die."

This is not stylistic. A refusal that explains its reasoning is argued *with*; a refusal emitting a rule identifier is routed *around*. Since the party encountering the refusal is frequently an agent that will attempt to satisfy the constraint, and secondarily an engineer deciding whether the gate is worth keeping, the message is part of the enforcement mechanism rather than a presentation concern.

The transcript in §4.1.1 is the clearest illustration. Nine refusals, each naming a consequence. An author reading that output learns what the decomposition is missing; an author reading nine rule identifiers learns only that the tool is in the way.

8.3 Redundant enforcement as a design pattern

Two predicates in this system are enforced twice by independent code over different data shapes: decision divergence (§4.1 and §4.2) and digest binding (verified at publication in §4.5, re-verified continuously in §4.4).

The reasoning generalises. A predicate enforced at a single point is a convention that holds until someone constructs a path around it - a new entry point, a migration script, a batch import. Enforced at two structurally different points, it becomes a property that a bypass must defeat twice, in two different ways. The cost is duplicated logic and the risk of divergence between the copies; the mitigation is that both are tested and both are fail-closed, so divergence manifests as an over-refusal rather than an under-refusal - a failure mode that is noticed.

8.4 Limitations

Single-filesystem assumption. Exclusivity depends on POSIX base system, `mkdir`, `flock`, and `link` atomicity. Not NFS-safe; not a distributed system.

The replay receipt confirms ingestion, not comprehension. An agent can echo a transport identifier and still misunderstand the instruction entirely. The mechanism closes T2 and makes no claim on T1 beyond it.

Falsifiability screening is lexical, with the precision and recall characteristics of §7.3 and the corpus-construction caveat stated there.

The threat model is primarily non-malicious. Process-lineage authentication raises the cost of impersonation without defeating a local attacker at equivalent privilege.

Ledger freshness is not mechanically enforced. The orchestrator-owned ledger gates self-scheduling, but nothing forces the orchestrator to keep it current - a stale ledger declaring a role active will sustain heartbeat wakes indefinitely.

Evaluation is $n=1$. All production measurement derives from a single deployment. The integrity result in particular reflects one team's binding discipline, and the 94% figure should be read as an existence proof that the signal-to-noise problem is severe and tractable, not as a population estimate.

8.5 Future work

Extending intent classification beyond two values: an append-only intent verifying that content grew without earlier bytes changing would cover log artifacts, which currently must be classified as snapshots and therefore lose tamper evidence entirely. Replacing lexical falsifiability screening with a model-based judge, evaluated against the survival argument of §8.1 rather than against accuracy alone. Extending process-lineage authentication to container and namespace identity for distributed deployments. And independent field measurement of the unfalsifiable-criteria population, which §7.3 could not obtain.

9. Conclusion

Multi-agent LLM systems are being given production authority on coordination layers built for cooperating processes. The failure modes that matter are not primarily adversarial: they are a confident agent proceeding on a wrong premise, an instruction that was written but never read, a success criterion that could never have failed, and an integrity signal that degraded into noise until someone switched it off.

None of these are model problems, and none are solvable by better prompting. They are properties of the coordination layer and are addressable there - by refusing to dispatch work whose success cannot be judged, by treating delivery as confirmed only on evidence the model itself produced, by classifying integrity

references so expected change and tampering are distinguishable, and by declining to reclaim a resource on a timeout alone.

The evaluation supports one claim strongly: reference-intent classification reduced an unusable 180-finding integrity signal to 11 actionable items, the difference between a control in use and a control in name. It supports a second more cautiously: lexical screening for unfalsifiable acceptance criteria achieves high precision on the class it targets, with the size of that class unmeasured.

The third finding is the one worth carrying away, and it was not sought. A commit adding a correct, well-tested control silently falsified the project's own verification claims while every test stayed green, because those claims lived in prose. The remediation then failed on its first attempt by making an assumption about the thing it verified - the same defect, one level up.

In systems built to mechanise assurance, the assurance claims are themselves the least mechanised artifact, and they fail silently. Any organisation deploying fail-closed controls should ask, of every claim in its documentation, what would break if this stopped being true - and then arrange for something to break.

References

A2A Project. (2026). *SEP: Capability-based authorization* [Discussion #1404]. GitHub.
<https://github.com/a2aproject/A2A/discussions/1404>

Crosby, S. A., & Wallach, D. S. (2009). *Efficient data structures for tamper-evident logging*. USENIX Security Symposium. https://static.usenix.org/event/sec09/tech/full_papers/crosby.pdf

ESAA: *Event sourcing for autonomous agents in LLM-based software engineering*. (2026). arXiv:2602.23193.
<https://arxiv.org/pdf/2602.23193>

Harnessing agent skills: Architectural patterns and a reference architecture for skill-mediated LLM agents. (2026). arXiv:2606.20631. <https://arxiv.org/html/2606.20631>

HearthNet: Edge multi-agent orchestration for smart homes. (2026). arXiv:2604.09618.
<https://arxiv.org/abs/2604.09618>

Lightweight tamper-evident log integrity verification for IoT edge environments: A Merkle tree pipeline with adaptive chunking. (2026). arXiv:2605.00065. <https://arxiv.org/pdf/2605.00065>

LLM-X: A scalable negotiation-oriented exchange for communication among personal LLM agents. (2026). arXiv:2605.11376. <https://arxiv.org/pdf/2605.11376>

PatchBoard: Schema-grounded state mutation for reliable and auditable LLM multi-agent collaboration. (2026). arXiv:2605.29313. <https://arxiv.org/pdf/2605.29313>

Saltzer, J. H., Reed, D. P., & Clark, D. D. (1984). End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4), 277–288.

SkillGuard: A permission framework for agent skills. (2026). arXiv:2606.03024. <https://arxiv.org/html/2606.03024>

Towards secure agent skills: Architecture, threat taxonomy, and security analysis. (2026). arXiv:2604.02837.
<https://arxiv.org/pdf/2604.02837>

Tran, K.-T., et al. (2025). *Multi-agent collaboration mechanisms: A survey of LLMs*. arXiv:2501.06322. <https://arxiv.org/pdf/2501.06322>

Tripwire. *File integrity monitoring: What it is and how it works*. <https://www.tripwire.com/state-of-security/file-integrity-monitoring>

Appendix A - Reproducing every result

Written for a reader starting from nothing. Every command below was executed on the environment in §A.1 before publication; where a result is *not* reproducible by a third party, that is stated rather than glossed.

A.1 Environment

Component	Version used
OS	Linux 5.14 (RHEL 9 family)
Python	3.9.25
PyYAML	5.4.1
make	GNU Make
git	any

Python 3.9 or later is required (`requires-python = ">=3.9"`). PyYAML is the only external dependency.

A.2 Clone and set up

```
git clone <coordination-substrate-url> coordination-substrate
cd coordination-substrate

# PyYAML is the only dependency. Either install the package:
python3 -m pip install -e .
# ...or just install the dependency and run from the source tree:
python3 -m pip install 'PyYAML>=5.4'
```

Installing the package additionally provides three console entry points (`cs-publish`, `cs-listen`, `cs-lease`). Nothing below requires them; every command uses `python3 -m` so it works from the source tree.

A.3 The test suite (§7.2.3)

```
make test
```

Expected:

```
Ran 140 tests in 11.753s
```

```
OK
```

Timing varies by host; the integration tests spawn real subprocesses. The count is the meaningful figure. `make lint` runs a compile check over the package and tests.

This suite includes `tests/test_documented_entry_points.py`, the control from §7.2.1. To see it work, break the documentation deliberately - remove `"credential_contract": {"mode": "none"}` from the `task-assigned` payload in `docs/WALKTHROUGH.md` - and re-run. It fails, naming the payload and the validator that refused it. Restore the field and it passes.

A.4 The publication demonstration (§4.5, §7.2.3)

```
make demo
```

Builds a bridge in a temporary directory, publishes through it, exercises three refusals and the lease lifecycle, and removes everything. Expect seven numbered steps ending `all demo assertions held`.

To reproduce the §7.2 *failure* rather than the fix, check out the commit immediately preceding the documented-entry-points test and run `make demo`; it exits non-zero with `assignment event requires credential_contract`.

A.5 The integrity mechanism (§4.4.1)

```
make integrity-demo
```

Builds a ledger carrying one `immutable_ref` binding and one `snapshot_at_publication` binding, edits each in turn, and prints the verifier's output after each. The two middle cases are the result: identical file modifications produce `fail_closed: false, exit 0` and `fail_closed: true, exit 1` respectively, determined solely by the intent declared at publication. The script asserts both outcomes, so it exits non-zero if the mechanism regresses.

A.6 The production integrity figures (§7.1) - **not third-party reproducible**

The 1,687-event / 1,977-binding measurement comes from a private production ledger that is not published. It cannot be reproduced externally, and no synthetic ledger would honestly stand in for it - the whole result concerns how bindings accumulate under real multi-agent operation.

Against any ledger you do have:

```
python3 -m coordination_substrate.integrity \
  --root /path/to/ledger --event-dir <events-dir> --format json
```

Or, if the ledger carries a topology document, via make:

```
make integrity ROOT=/path/to/ledger
```

`--strict` returns exit 1 when `fail_closed` is true, which is how the check composes into CI. `--format markdown` (the default) renders the triage summary shown in §4.4.1.

A.7 Dispatch admission (§4.1.1)

The validator ships in the skills repository:

```
git clone <ai-skills-url> ai-skills
cd ai-skills/skills/mission-decompose/references
```

Save the two decomposition documents from §A.8, then:

```
python3 decompose_check.py refused.json    ; echo "exit: $?"    # nine refusals,
exit 1
python3 decompose_check.py issuable.json    ; echo "exit: $?"    # issuable,
exit 0
```

`refused.json` differs from `issuable.json` in exactly the nine fields the transcript names. The capability vocabulary defaults to the shipped `capabilities.json`; supply your own with `--capabilities FILE`.

The skills repository's own suite runs with:

```
./ci.sh          # compile check, install.sh syntax, then 269 template tests
```

A.8 The two decomposition documents

`refused.json` - nine deliberate defects:

```
{
  "layer": "platform-build",
  "mission": {
    "who": "the platform team",
    "what": "harden the coordination bridge",
    "when": "this sprint",
    "where": "the mission-platform repository"
  },
  "intent": {
    "purpose": "reduce coordination failures",
    "key_tasks": ["harden the publisher"],
```

```

    "end_state": "the bridge is reliable"
  },
  "terrain": [
    {"fact": "the bridge has 1687 events",
     "confirmed_by": "ls coordination/bridge/events | wc -l", "observed": "1687"},
    {"fact": "CI runs on every push"}
  ],
  "tasks": [
    {"id": "T-1", "type": "specified", "essential": true,
     "statement": "Add credential contract validation to the publisher",
     "acceptance": "the validator works correctly and handles malformed input gracefully",
     "capabilities": ["read-repo", "run-tests"]},
    {"id": "T-2", "type": "specified", "essential": true,
     "statement": "Improve publisher performance",
     "acceptance": "performance is improved",
     "capabilities": ["read-repo"]}
  ],
  "limits": {
    "constraints": ["run the full test suite"],
    "restraints": ["run the full test suite", "force push"]
  },
  "ccir": [
    {"requirement": "Does the ledger contain duplicate identities?",
     "outcomes": [
       {"outcome": "yes", "next_action": "investigate the ledger"},
       {"outcome": "no", "next_action": "investigate the ledger"}
     ]
    }
  ],
  "time": {"total_minutes": 120, "planning_minutes": 90},
  "backbrief_required": false
}

```

`issuable.json` - the same decomposition with all nine corrected:

```

{
  "layer": "platform-build",
  "mission": {
    "who": "the platform team",
    "what": "harden the coordination bridge",
    "when": "this sprint",
    "where": "the mission-platform repository",
    "why": "so an unattended agent cannot publish an assignment that omits credential handling"
  },
  "intent": {
    "purpose": "reduce coordination failures reaching production",
    "key_tasks": ["harden the publisher", "prove the documented path still runs"],
    "end_state": "no assignment publishes without a credential contract, and CI fails if the walkthrough stops working"
  },
  "terrain": [

```



```

    {"fact": "the ledger holds 1687 events",
     "confirmed_by": "python3 -m coordination_substrate.integrity --format json",
     "observed": "loaded: 1687"},
    {"fact": "the suite is green at 131 tests",
     "confirmed_by": "make test", "observed": "Ran 131 tests ... OK"}
],
"tasks": [
    {"id": "T-1", "type": "specified", "essential": true,
     "statement": "Require a credential contract on every assignment event",
     "acceptance": "publishing an assignment without credential_contract exits
non-zero with 'assignment event requires credential_contract'",
     "capabilities": ["read-repo", "run-tests"]},
    {"id": "T-2", "type": "implied", "essential": true,
     "statement": "Assert the documented entry points still execute",
     "acceptance": "scripts/demo.py exits 0 and every walkthrough payload matches
its documented accept/refuse outcome",
     "capabilities": ["read-repo", "run-tests"]}
],
"limits": {
    "constraints": ["run the full test suite before publishing"],
    "restraints": ["force push", "amend published events"]
},
"ccir": [
    {"requirement": "Does the walkthrough still publish successfully?",
     "outcomes": [
         {"outcome": "yes", "next_action": "proceed to publication"},
         {"outcome": "no", "next_action": "update the walkthrough and re-run the
suite"}
     ]
    }
],
"time": {"total_minutes": 180, "planning_minutes": 40},
"backbrief_required": true
}

```

A.9 Falsifiability screen precision and recall (§7.3)

The harness imports the corpus directly from the validator, so it measures the shipped patterns rather than a copy:

```

#!/usr/bin/env python3
"""Measure the falsifiability screen against a labelled corpus.
Run from skills/mission-decompose/references/."""
import importlib.util
from pathlib import Path

spec = importlib.util.spec_from_file_location("dc", "decompose_check.py")
dc = importlib.util.module_from_spec(spec)
spec.loader.exec_module(dc)

def screened(text):

```

```

for pattern, label in dc.UNFALSIFIABLE:
    if pattern.search(text):
        return label
return None

UNFALSIFIABLE = [
    "the validator works correctly", "the endpoint works properly",
    "retries work as expected", "the parser works fine", "output is good",
    "the logs are clean", "the design is reasonable", "results look sensible",
    "the config is fine", "retry as appropriate", "scale as needed",
    "log as necessary", "handles errors gracefully",
    "handles malformed input properly", "handles the edge case appropriately",
    "no major issues", "no significant problems", "no obvious regressions",
    "latency improved", "throughput is improved",
]
FALSIFIABLE = [
    "publishing without credential_contract exits non-zero",
    "GET /health returns 200 within 50ms at p99",
    "make test reports 140 passed, 0 failed",
    "the pointer sha256 equals sha256 of the installed file",
    "integrity --strict exits 1 when fail_closed is true",
    "the lease is refused with 'lease owner tuple differs'",
    "scripts/demo.py exits 0 and prints 'all demo assertions held'",
    "duplicate (writer, sequence) count is 0",
    "the walkthrough payload set validates in both directions",
    "p95 publish latency is below 20ms over 1000 publishes",
    "error rate improved by 40% relative to the 2026-07-01 baseline",
    "the build produces a single artifact under 5MB",
    "unparseable event count is 0",
    "reap declines with reason owner-alive while the owner runs",
    "the child stderr log is non-empty after a failed start",
    "compileall reports no syntax errors",
    "every essential task has a non-empty acceptance field",
    "the corpus flags 'works correctly' and not 'works around the lock'",
    "snapshot_drift is reported but does not set fail_closed",
    "the listener state file shows status running after receipt",
]

tp = sum(1 for s in UNFALSIFIABLE if screened(s))
fp = sum(1 for s in FALSIFIABLE if screened(s))
print(f"recall    = {tp / len(UNFALSIFIABLE):.2f}")
print(f"precision = {tp / (tp + fp):.2f}")
for s in FALSIFIABLE:
    if screened(s):
        print(f"FALSE POSITIVE: {screened(s)!r} <- {s}")

```

Expected output:

```

recall    = 1.00
precision = 0.95

```

FALSE POSITIVE: 'works correctly' <- the corpus flags 'works correctly' and not 'works around the lock'

The false positive is deterministic and is discussed in §7.3.

Appendix B - Enumerated technical disclosure

Each of the following is placed in the public domain as of the publication date, individually and in every combination.

1. Refusing to dispatch work to an autonomous agent where a natural-language acceptance criterion for an essential task matches a stored corpus of unfalsifiable constructions.
2. Refusing to dispatch where a declared information requirement's outcomes do not map to at least two distinct subsequent actions.
3. Refusing to dispatch where an asserted fact about system state lacks either a confirming command or an observation obtained from it.
4. Evaluating such predicates without executing any command named in the evaluated document.
5. Confirming delivery to a language-model agent only upon detecting an embedded identifier within content generated by that model, withholding advancement of a delivery watermark until then.
6. Gating all self-originated agent wake conditions on state owned by a distinct orchestrator.
7. Declaring, per reference and at publication time, whether a recorded digest binding must always verify or records a point-in-time snapshot; storing that declaration inside the append-only record; and excluding snapshot mismatches from a composite fail-closed indicator.
8. Reporting a binding whose declared intent is unrecognised as itself a defect.
9. Computing a pointer's digest from bytes as installed and refusing any caller-supplied digest.
10. Ordering publication so failure can produce an unreferenced record but never a pointer to absent or mismatched content.
11. Authenticating a publisher by requiring a registered listener's child process to appear in the calling process's ancestry.
12. Returning a replay indication rather than a second grant where an exclusive claim is re-presented with an identical identity tuple.
13. Reclaiming an expired exclusive claim only upon affirmative observation that the recorded owner process is absent, with expiry alone never sufficient.
14. Identifying a claim owner by process identifier paired with process start time, omitting rather than nulling that field where unobtainable, such that a claim lacking it is never reclaimed.